

Network Programming

- OOB -

Lenuta Alboaie
adria@info.uaic.ro

Content

- Sending and receiving data using *out-of-band* mechanism

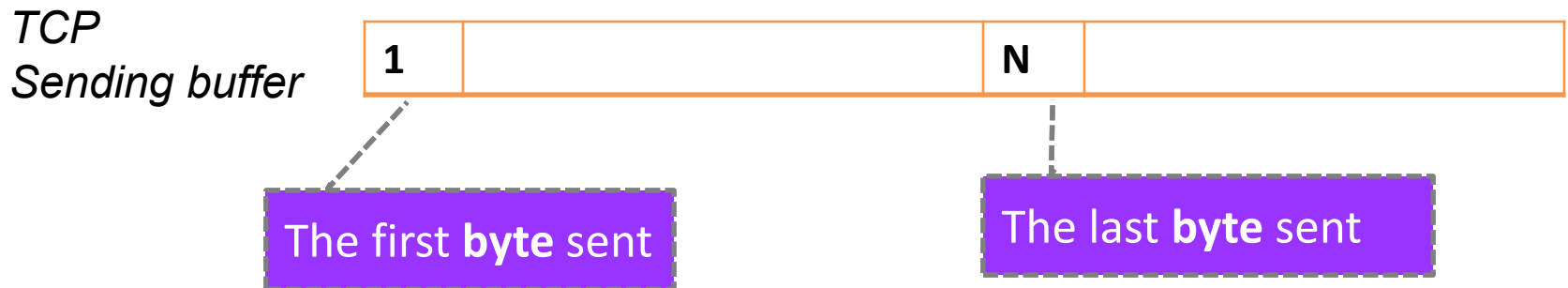
Sending and receiving data using out-of-band mechanism

- The idea: during a connection, when data are transmitted (“**in-band data**”), and at one end “something” happens it is necessary to send a priority notification (“**out-of-band data**”) to the other peer
- The achieving mechanism:
 - It uses the URG bit in TCP header
 - The TCP header contains a field indicating the location of the urgent data to be sent
 - **OOB data sent:**
 - To send an urgent byte in a data stream **send()** primitive may be used:

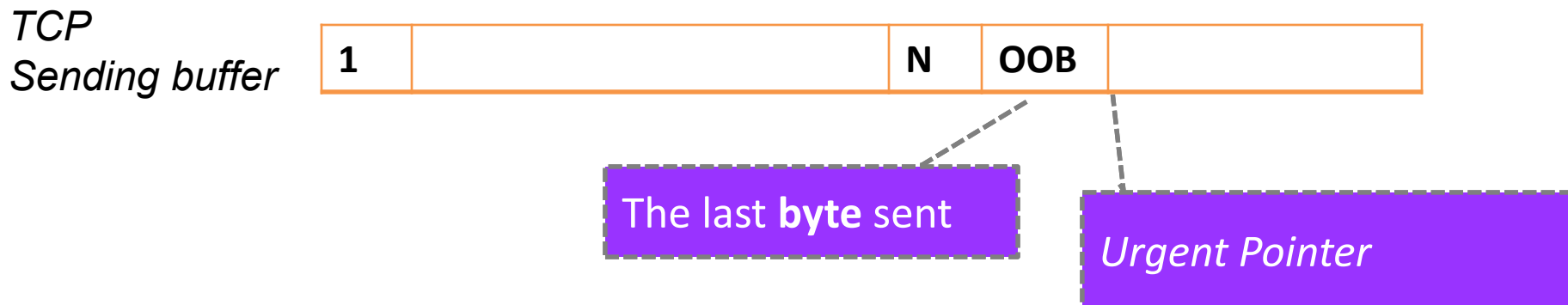
send (sd, buff, 1, **MSG_OOB**);

Sending and receiving data using out-of-band mechanism

Sending an OOB byte



send (sd, "a", 1, **MSG_OOB**) Sending an OOB
byte



Sending and receiving data using out-of-band mechanism

Receiving OOB data :

- **SIGURG** signal is generated
- **select()** call will modify the exceptional list descriptors

OOB data reading:

- If the socket has no `SO_OOBINLINE` option, the OOB message is placed in a special buffer (*out-of-band buffer*); reading the buffer can be done with **recv()** or **recvmsg()** setting **MSG_OOB**
- If the socket has `SO_OOBINLINE` option, the OOB message is placed in the normal reception buffer;
- The process will know that it reached *the special* byte depending on *out of-band-mark* value associated with the connection

Sending and receiving data using out-of-band mechanism

`socketmark()` call

- When OOB data is received, `out-of-band-mark` association is performed – and represents the OOB position in the data stream (sent by the transmitter)
- `socketmark()` ensures that the receiver will determine whether *out-of-band mark* is associated or not with the connection

```
#include <sys/socket.h>
```

```
int socketmark (int sockfd) ;
```

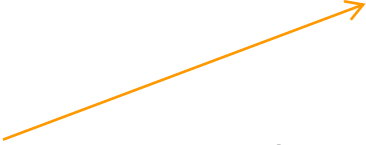
Returns: 1 – is *out-of-band mark*,

0 – is no *out-of-band mark*

–1 on error

Sending and receiving data using out-of-band mechanism

socketatmark() – observations

- *out-of-band mark* applies whether the process receives data in *out-of-band inline* mode (socket with `SO_OOBINLINE` option) or in an *out-of-band* mode (`MSG_OOB` flag)
- It is implemented using `ioctl()` and `SIOCATMARK` int value;


```
error = ioctl(tcp_socket, ioctl_type, &value);
```

Sending and receiving data using out-of-band mechanism

socketmark() – discussions & example

- If you have received *OOB inline* data, `socketmark()` returns true if the next readable byte was sent with MSG_OOB flag
- If the *socket* has no SO_OOBINLINE option, `socketmark()` return *true* if the next readable byte is the first byte sent after OOB data
- The read operation stops according to *out of-band-mark*
 - Example:

If there are 100 bytes in buffer, but just 5 bytes up to *out of-band-mark*, even if the process reads 100 bytes, it will initially receive only the first 5 bytes

Sending and receiving data using out-of-band mechanism

Possible errors:

- OOB data is expected to be read, but they have not yet been sent – it returns **EINVAL**
- The process has been notified that it will receive OOB data(via **select()** or **SIGURG**), he tries to read but they haven't reached yet – it returns **EWOULDBLOCK**
- It tries to read the same **OOB** byte multiple times – it returns **EINVAL**
- If the **SO_OOBINLINE** is set, but the reading is performed using **MSG_OOB** flag – it returns **EINVAL**

Sending and receiving data using out-of-band mechanism

- Uses:
 - Situations when one endpoint signals an exceptional circumstance to the peer endpoint (even when flow control has stopped the sender)
 - To detect early some communication errors between client and the server (*heart-beat*)

Sending and receiving data using out-of-band mechanism

OOB implementation using SIGURG:

```
if (listen (sd, 5) == -1)  { ... }
...
while (1)
{
    ... client = accept (sd, (struct sockaddr *) &from, &len);
    signal(SIGURG, urgHandler);
    fcntl(client, F_SETOWN, getpid()); /*setarea proprietarului socketului conectat */
    for(;;)  {
        if((n=read(client,msg,sizeof(msg)-1))==0)
        {
            printf("Am primit EOF\n");
            break;
        }
        else
        {
            msg[n]=0;
            printf("Am citit %d octeti: %s\n",n, msg);
        }
    } ... } //while
void urgHandler(int signnr)
{
    int n;          char buff[100];
    printf("SIGURG e primit\n");
    n=recv(client,buff, sizeof(buff)-1, MSG_OOB);
    buff[n]='\0';
    printf("Am citit %d octet OOB %s\n",n, buff);
}
```

Sending and receiving data using out-of-band mechanism

OOB implementation using `socketmark()`

DEMO

- **The transmitter** sends 4 bytes of normal data, then an OOB byte and then another normal byte
- **The receiver** doesn't use SIGURG or select(); he calls `socketmark()` primitive;

Summary

- Sending and receiving data using *out-of-band* mechanism

Bibliography

- UNIX Network Programming: The sockets networking API, W. Richard Stevens, Bill Fenner, Andrew M. Rudoff
- The Illustrated Network: How TCP/IP Works in a Modern Network (The Morgan Kaufmann Series in Networking), Walter Goralski



Questions?